

Autores

Dave Bergmann

Senior Staff Writer, AI Models

IBM Think

Cole Stryker

Staff Editor, AI Models

IBM Think

O que é embedding vetorial?

Embeddings vetoriais são representações numéricas de pontos de dados que expressam diferentes tipos de dados, incluindo dados não matemáticos como palavras ou imagens, como um array de números que modelos de [aprendizado de máquina](#) (ML) podem processar.

[Modelos de inteligência artificial \(IA\)](#), desde algoritmos simples de [regressão linear](#) até as complexas [redes neurais](#) usadas em [deep learning](#), operam através de lógica matemática. Qualquer dado em que um modelo de IA opera, incluindo dados não estruturados como texto, áudio ou imagens, deve ser expresso numericamente. O embedding vetorial é uma maneira de converter um ponto de dado não estruturado em um array de números que ainda expressa o significado original desse dado.

[Treinar modelos](#) para produzir representações vetoriais de pontos de dados que correspondam significativamente às suas funcionalidades do mundo real nos permite fazer suposições úteis sobre como as embeddings vetoriais se relacionam entre si. Intuitivamente, quanto mais semelhantes são dois pontos de dados do mundo real, mais semelhantes devem ser suas respectivas embeddings vetoriais. Funcionalidades ou qualidades compartilhadas por dois pontos de dados devem ser refletidas em ambas as suas embeddings vetoriais. Pontos de dados dissimilares devem ter embeddings vetoriais dissimilares.

Munidos de tais suposições lógicas, os embeddings vetoriais podem ser usados como inputs para modelos que realizam tarefas úteis no mundo real por meio de operações matemáticas que comparam, transformam, combinam, ordenam ou manipulam essas representações numéricas.

Expressar pontos de dados como vetores também permite a interoperabilidade de diferentes tipos de dados, atuando como uma espécie de *língua franca* entre diferentes formatos de dados ao representá-los no mesmo espaço de embeddings. Por exemplo, assistentes de voz de smartphones "traduzem" as entradas de áudio do usuário em embeddings vetoriais, que por sua vez usam embeddings vetoriais para o [processamento de linguagem natural \(NLP\)](#) dessa entrada.

Assim, as embeddings vetoriais sustentam quase todo o aprendizado de máquina moderno, alimentando modelos usados nos campos de NLP e computer vision, e servindo como os blocos de construção fundamentais da IA generativa.

O que é um vetor?

Vetores pertencem à categoria maior de *tensores*. No aprendizado de máquina (ML), "tensor" é usado como um termo genérico para uma matriz de números (ou uma matriz de matrizes de números) em um espaço n -dimensional, funcionando como um dispositivo matemático de registro de dados.

É útil notar que certas palavras são usadas de forma diferente em um contexto de ML do que na linguagem cotidiana ou em outros contextos matemáticos. "Vetor" em si, por exemplo, tem uma conotação mais específica em física (onde geralmente se refere a uma quantidade com magnitude e direção) do que em ML.

Da mesma forma, a palavra "dimensão" tem implicações diferentes em ML, dependendo de seu contexto. Ao descrever um tensor, refere-se a quantas matrizes esse tensor contém. Ao descrever um vetor, refere-se a quantos componentes (números individuais) esse vetor contém. Termos análogos como "ordem" ou "grau" podem ajudar a reduzir a ambiguidade.

- Um *escalar* é um tensor de zero dimensão, contendo um único número. Por exemplo, um sistema que modela dados meteorológicos pode representar a temperatura máxima de um único dia (em graus Celsius) na forma escalar como 33 .
- Um *vetor* é um tensor unidimensional (ou *de primeiro grau* ou *de primeira ordem*), contendo múltiplos escalares do mesmo tipo de dados. Por exemplo, o modelo meteorológico pode representar as temperaturas mínima, média e máxima daquele dia em forma vetorial, como (25, 30, 33) . Cada componente escalar é uma *funcionalidade* (isto é, uma dimensão) do vetor, correspondendo a uma funcionalidade do clima desse dia.
- Uma *tupla* é um tensor de primeira ordem contendo escalares de mais de um tipo de dados. Por exemplo, o nome, a idade e a altura (em centímetros) de uma pessoa podem ser representados em forma de tupla como (Jane, Smith, 31, 65) .
- Uma *matriz* é um tensor bidimensional (ou *de segunda ordem* ou *segundo grau*), contendo múltiplos vetores do mesmo tipo de dado. Pode ser visualizada intuitivamente como uma grade bidimensional de escalares na qual cada linha ou coluna é um vetor. Por exemplo, esse modelo meteorológico pode representar todo o mês de junho como uma matriz 3x30, na qual cada linha é um vetor de recursos descrevendo as temperaturas mínima, média e máxima de um dia individual.
- Tensores com três ou mais dimensões, como os [tensores tridimensionais usados para representar imagens coloridas](#) em algoritmos de visão computacional, são chamados de *matrizes multidimensionais* ou tensores N -dimensionais.

Várias transformações simples também podem ser aplicadas a matrizes ou outros tensores n -dimensionais para representar os dados que contêm em forma vetorial. Por exemplo, uma matriz 4x4 pode ser achatada em um vetor de 16 dimensões; um tensor tridimensional de uma

imagem de 4x4 pixels pode ser achatado em um vetor de 48 dimensões. As embeddings predominantemente assumem a forma de vetores na ML moderna.

Vetores versus embeddings:

Embora os termos sejam frequentemente usados de forma intercambiável em ML, "vetores" e "embeddings" não são necessariamente a mesma coisa.

Uma *integração* é qualquer representação numérica de dados que capture suas qualidades relevantes de forma que os algoritmos de ML possam processá-los. Os dados são *integrados* em um espaço n -dimensional.

Em teoria, os dados não *precisam* ser incorporados (embedded) como um vetor, especificamente. Por exemplo, alguns tipos de dados podem ser integrados na forma de tuplas.¹ Mas, na prática, as embeddings predominantemente assumem a forma de vetores na ML moderna.

Por outro lado, vetores em outros contextos, como a física, não são necessariamente embeddings. Mas em ML, vetores geralmente são embeddings e embeddings geralmente são vetores.

Como funciona o embedding vetorial?

Uma embedding vetorial transforma um ponto de dados, como uma palavra, frase ou imagem, em uma matriz n -dimensional de números que representam as características desse ponto de dados — suas *funcionalidades*. Isso é alcançado treinando um *modelo de embedding* em um grande conjunto de dados relevante para a tarefa em questão ou usando um modelo pré-treinado.

Para entender embeddings vetoriais, é necessária a explicação de alguns conceitos-chave:

- **Como os embeddings vetoriais representam dados.**
- **Como os embeddings vetoriais podem ser comparados.**
- **Como modelos podem ser usados para gerar embeddings vetoriais.**

Como os embeddings vetoriais representam dados

No aprendizado de máquina, as "dimensões" dos dados não se referem às dimensões familiares e intuitivas do espaço físico. No espaço vetorial, cada dimensão corresponde a uma *funcionalidade individual dos dados*, da mesma forma que comprimento, largura e profundidade são *funcionalidades* de um objeto no espaço físico.

As embeddings vetoriais normalmente lidam com *dados de alta dimensão* porque, na prática, a maioria das informações não numéricas é de alta dimensão. Por exemplo, mesmo uma pequena e simples imagem em preto e branco de 28x28 pixels de um dígito manuscrito do [conjunto de dados MNIST](#) pode ser representada como um vetor de 784 dimensões, no qual cada dimensão corresponde a um pixel individual, cujo valor em escala de cinza varia de 0 (para preto) a 1 (para branco).

No entanto, nem todas essas dimensões dos dados conterão informações úteis. Em nosso exemplo do MNIST, o dígito real em si representa apenas uma pequena fração da imagem: o restante é um fundo em branco, ou "ruído". Portanto, seria mais preciso dizer que estamos "integrando (embedding) uma representação da imagem em um espaço de 784 dimensões" do que dizer que estamos "representando 784 diferentes funcionalidades da imagem."

Embeddings vetoriais eficientes de dados de alta dimensão frequentemente envolvem algum grau de [redução de dimensionalidade](#): a compactação de dados de alta dimensão para um [espaço de menor dimensão que omite informações irrelevantes ou redundantes](#).

A redução de dimensionalidade aumenta a velocidade e a eficiência do modelo, embora com um comprometimento potencial da precisão ou exatidão, porque vetores menores requerem menos recursos computacionais para operações matemáticas. Também pode ajudar a diminuir o risco de [overfitting](#) dos dados de treinamento. Diferentes métodos de redução de dimensionalidade, como [autocodificadores](#), [convoluções](#), [análise de componentes principais](#) e embedding de vizinhos estocásticos distribuídos em T (t-SNE), são mais adequados para diferentes tipos de dados e tarefas.

Enquanto as dimensões dos dados vetoriais de imagens são relativamente objetivas e intuitivas, determinar as funcionalidades relevantes de algumas modalidades de dados, como os significados semânticos e as relações contextuais da linguagem, é mais abstrato ou subjetivo. Nesses casos, as características específicas representadas pelas dimensões das embeddings vetoriais podem ser estabelecidas por meio de uma [engenharia de funcionalidades](#) manual ou, mais comumente na era do deep learning, determinadas implicitamente por meio do processo de treinar um modelo para fazer previsões precisas.

Como comparar incorporações vetoriais

A lógica central das embeddings vetoriais é que embeddings n -dimensionais de pontos de dados semelhantes devem estar agrupados próximas no espaço n -dimensional. No entanto, as embeddings podem ter dezenas, centenas ou até milhares de dimensões. Isso vai muito além dos espaços bidimensionais ou tridimensionais nos quais nossas mentes podem visualizar intuitivamente as coisas "próximas" umas das outras.

Em vez disso, uma das várias medidas matemáticas pode ser usada para inferir a semelhança relativa ou proximidade de diferentes embeddings vetoriais. A melhor medida de similaridade para uma situação específica depende em grande parte da natureza dos dados e do propósito das comparações.

- A **distância euclidiana** mede a distância média em linha reta entre os pontos correspondentes de diferentes vetores. A diferença entre dois vetores n -dimensionais $\mathbf{a}$ e $\mathbf{b}$ é calculada primeiro adicionando os quadrados das diferenças entre cada um de seus componentes correspondentes — ou seja, $(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2$ — e, em seguida, extraindo a raiz quadrada dessa soma. Como a distância euclidiana é sensível à magnitude, é útil para dados que reflitam coisas como tamanho ou contagens. Os valores variam de 0 (para vetores idênticos) até ∞ .
- A **distância de cosseno**, também chamada de *similaridade de cosseno*, é uma medida normalizada do cosseno do ângulo entre dois vetores. A distância de cosseno varia de -1 a 1, onde 1 representa vetores idênticos, 0 representa vetores *ortogonais* (ou não

relacionados), e -1 representa vetores totalmente opostos. A similaridade de cosseno é amplamente usada em tarefas de NLP porque normaliza naturalmente as magnitudes vetoriais, o que a torna menos sensível à frequência relativa das palavras nos dados de treinamento do que a distância euclidiana.

- **Produto escalar** é, algebricamente falando, a soma do produto dos componentes correspondentes de cada vetor. Geometricamente falando, é uma versão não normalizada da distância de cosseno que também reflete frequência ou magnitude.

Modelos de embedding

Modelos de embeddings independentes podem ser pré-treinados ou treinados a partir do zero em tarefas ou dados de treinamento específicos. Cada forma de dados normalmente se beneficia de uma arquitetura de redes neurais específica, mas o uso de um algoritmo específico para uma tarefa específica costuma ser uma "melhor prática" em vez de uma regra explícita.

Em alguns cenários, o processo de embedding é parte integrante de uma rede neural maior. Por exemplo, em [redes neurais convolucionais \(CNNs\)](#) do tipo codificador-decodificador, utilizadas em tarefas como [segmentação de imagens](#), otimizar toda a rede para gerar previsões precisas exige treinar as camadas do codificador para produzir embeddings vetoriais eficazes das imagens de entrada.

Modelos pré-treinados

Para muitos casos de uso e campos de estudo, modelos pré-treinados oferecem integrações úteis que podem ser usadas como entradas para modelos personalizados ou bancos de dados de vetores. Esses modelos de código aberto geralmente são treinados em um conjunto massivo e diversificado de dados para aprender integrações úteis para muitas tarefas subsequentes, como [aprendizado com poucos exemplos](#) ou [aprendizado sem exemplos](#).

Para dados textuais, modelos básicos de [integração de palavras](#) de código aberto, como o Word2Vec do Google ou os Global Vectors (GloVe) da Universidade de Stanford, podem ser treinados do zero, mas também são disponibilizados em variantes pré-treinadas com dados públicos, como Wikipedia e Common Crawl. Da mesma forma, grandes modelos de linguagem (LLMs) do tipo codificador-decodificador, frequentemente utilizados para integrações, como o BERT e suas diversas variantes, são pré-treinados com uma enorme quantidade de dados textuais.

Para tarefas de visão computacional, modelos pré-treinados de classificação de imagens, como ImageNet, ResNet ou VGG, podem ser adaptados para produzir embeddings simplesmente removendo sua camada final de predição totalmente conectada.

Modelos de integração personalizados

Alguns casos de uso, especialmente aqueles que envolvem conceitos complexos ou novas categorias de dados, se beneficiam do [ajuste fino](#) de modelos pré-treinados ou do treinamento de modelos de integração totalmente personalizados.

Os domínios jurídico e médico são exemplos proeminentes de campos que frequentemente dependem de vocabulário, bases de conhecimento ou imagens difíceis e altamente

especializadas que provavelmente não foram incluídas nos dados de treinamento de modelos mais generalistas. Complementar o conhecimento base de modelos pré-treinados através de treinamento adicional em exemplos específicos do domínio pode ajudar o modelo a produzir embeddings mais eficazes.

Embora isso também possa ser alcançado projetando uma arquitetura de rede neural personalizada ou treinando uma arquitetura conhecida do zero, fazê-lo requer recursos e conhecimento institucional que podem estar fora do alcance da maioria das organizações ou entusiastas.

Embedding vetorial para imagens

Embeddings de imagens convertem informações visuais em vetores numéricos usando os valores dos pixels de uma imagem para corresponder aos componentes do vetor. Eles geralmente dependem de CNNs, embora nos últimos anos os modelos de visão computacional tenham utilizado cada vez mais redes neurais baseadas em transformadores².

Imagens com um esquema de cores RGB típico são numericamente representadas como uma matriz tridimensional, na qual essas três matrizes correspondem aos respectivos valores vermelho, verde e azul de cada pixel. Imagens RGB são geralmente de 8 bits, o que significa que cada valor de cor para um pixel pode variar de 0 a 256 (ou 2^8). Como descrito anteriormente, imagens em preto e branco são numericamente representadas como uma matriz bidimensional de pixels em que cada pixel tem um valor entre 0 e 1.

As [convoluções](#) usam filtros numéricos bidimensionais, chamados *kernels*, para extrair recursos da imagem. Os pesos dos kernels mais propícios para extrair recursos relevantes são, eles mesmos, um parâmetro aprendível durante o treinamento do modelo. Essas convoluções produzem um mapa de características da imagem.

Quando necessário, o *padding* é usado para manter o tamanho original do input adicionando camadas extras de zeros às linhas e colunas externas do array. Por outro lado, o *pooling*, que essencialmente resume recursos visuais tomando apenas seus valores mínimos, máximos ou médios, pode ser usado para uma redução adicional de dimensionalidade.

Finalmente, a representação compactada é, então, *achatada* em um vetor.

Busca de imagens

Uma aplicação intuitiva de embedding de imagens é a *busca de imagens*: um sistema que recebe dados de imagens como entrada e retorna outras imagens com embeddings vetoriais similares, como um aplicativo de smartphone, que identifica uma espécie de planta a partir de uma fotografia.

Uma execução mais complexa é a pesquisa *multimodal* de imagens, tomando o texto como entrada e retornando imagens relacionadas a esse texto. Isso não pode ser realizado simplesmente tomando um embedding de texto de um modelo de linguagem e usando-o como entrada para um modelo de computer vision separado. Em vez disso, os dois modelos de embeddings devem ser explicitamente treinados para se correlacionar entre si.

Um algoritmo proeminente usado tanto para embeddings de imagem quanto de texto é o *contrastive language-image pretraining* (CLIP), desenvolvido originalmente pela OpenAI. O

CLIP foi treinado em um enorme conjunto de dados não rotulados de mais de 400 milhões de pares de imagem-legenda retirados da internet. Esses pares foram usados para treinar conjuntamente um codificador de imagem e um codificador de texto do zero, usando *perda contrastiva* para maximizar a similaridade de cosseno entre embeddings de imagem e os embeddings de suas legendas correspondentes.

Geração de imagens

Outra aplicação importante para a embedding de imagens é a *geração de imagens*: a criação de novas imagens.

Um método para gerar novas imagens a partir de embeddings de imagens utiliza [autocodificadores variacionais \(VAEs\)](#). Os VAEs codificam duas embeddings vetoriais diferentes de entrada: um vetor de *médias* e um vetor de *desvios padrão*. Ao amostrar aleatoriamente a distribuição de probabilidade que essas embeddings vetoriais representam, os VAEs podem usar sua rede de decodificadores para gerar variações desses dados de entrada.

Um método de geração de imagens baseado em embeddings mais proeminente, especialmente nos últimos anos, usa o algoritmo CLIP mencionado anteriormente. Modelos de síntese de imagens, como Do ALL-E, Midjourney e Stable Diffusion, recebem prompts de texto como entrada, usando o CLIP para embutir uma representação vetorial do texto; essa mesma embedding vetorial, por sua vez, é usada por um [modelo de difusão](#) para essencialmente reconstruir uma nova imagem.

Embedding vetorial para PLN

Embeddings de texto são menos diretos. Eles devem representar numericamente conceitos abstratos, como significado semântico, conotações variáveis e relações contextuais entre palavras e frases. Simplesmente representar palavras em termos de suas letras, da mesma forma que embeddings de imagens representam visuais em termos de valores de pixels, não produziria embeddings significativos.

Enquanto a maioria dos modelos de computer vision é treinada utilizando [aprendizado supervisionado](#), os modelos de embeddings para NLP demandam [aprendizado autossupervisionado](#) com uma quantidade realmente maciça de dados de treinamento, para captar adequadamente os diversos significados possíveis da linguagem em diferentes contextos.

Os embeddings resultantes alimentam muitas das tarefas comumente associadas à IA generativa, desde tradução de idiomas até chatbots conversacionais, resumo de documentos e serviços de perguntas e respostas.

Modelos de embedding de texto

Os modelos usados para gerar embeddings vetoriais para dados de texto geralmente não são os mesmos usados para gerar texto real.

Os populares LLMs amplamente utilizados para geração de texto e outras tarefas de IA generativa, como os modelos ChatGPT ou [Llama](#), são modelos apenas de decodificação *autorregressivos*, também conhecidos como *modelos de linguagem causal*. No treinamento, eles são apresentados à primeira palavra de uma amostra de texto e têm a tarefa de prever continuamente a próxima palavra até o final da sequência. Embora isso seja

adequado para aprender a gerar texto coerente, não é ideal para aprender embeddings vetoriais úteis independentes.

Em vez disso, embeddings de texto tipicamente dependem de *modelos de linguagem mascarada*, como representações de codificador bidirecional de transformadores (BERT), lançado pela primeira vez em 2018. No treinamento, esses modelos de codificador-decodificador recebem sequências de texto com determinadas palavras *mascaradas* (ocultas) e têm a tarefa de preencher os espaços em branco. Esse exercício valoriza embeddings que capturam melhor as informações sobre uma palavra ou frase específica e sua relação com o contexto ao redor. O Word2vec realiza uma tarefa de treinamento semelhante, embora utilize uma arquitetura mais simples de rede neural com duas camadas.

Em junho de 2024, BERT continua sendo o modelo de linguagem mais popular no Hugging Face, com mais de 60 milhões de downloads no mês anterior.³ Várias variantes proeminentes do [BERT](#) foram adaptadas para tipos específicos de integrações de linguagem e cenários:

- **SBERT:** Também conhecido como sentence BERT e sentence transformers, o SBERT é uma variante do BERT com uma estrutura de [rede neural siamesa](#) adaptada, [ajustada](#) em pares de sentenças para aprimorar sua capacidade de codificar integrações de sentenças.
- **DistilBERT:** uma variante leve do BERT, criada através da [destilação de conhecimento](#) do modelo base do BERT em um modelo menor que roda 60% mais rápido, mantendo mais de 95% do desempenho do BERT em algumas métricas⁴.
- **RoBERTa:** abreviação de "abordagem de pré-treinamento do BERT otimizado de forma robusta", o RoBERTa refinou o procedimento de treinamento do BERT para otimizar seu desempenho.

Tipos de embeddings de texto

Embeddings vetoriais podem ser usados para representar vários dados de linguagem natural.

Embeddings de palavras

[Embeddings de palavras](#) visam capturar não apenas o significado semântico de palavras individuais, mas também sua relação contextual com outras palavras com as quais frequentemente coocorrem. Ao fazer isso, embeddings de palavras podem generalizar bem para novos contextos e até mesmo palavras raras ou previamente desconhecidas.

O GloVe, um modelo popular de embedding de palavras, foi treinado em uma "matriz global de coocorrência palavra-palavra", inferindo significado semântico e relacionamentos semânticos a partir de com que frequência palavras específicas são usadas próximas umas das outras. Por exemplo, o significado pode derivar de como "gelo" e "vapor" coincidem com "água" aproximadamente com a mesma frequência, mas coincidem com "sólido" e "gás" em proporções muito diferentes.⁵

A forma como as dimensões de um vetor de embedding de palavras capturam implicitamente esses relacionamentos nos permite manipulá-los matematicamente de maneiras úteis e intuitivas. Em um esquema de embedding de palavras bem configurado, subtrair o vetor de "homem" do vetor de "rei" e adicionar o vetor de "mulher" deve essencialmente resultar no vetor de "rainha".

As integrações de sentenças

representam o significado semântico de frases ou sentenças inteiras, em vez de palavras individuais. Elas geralmente são geradas pelo SBERT ou por outras variantes de transformadores de frases.

- Embeddings de frases podem incorporar representações de consultas de usuários, para uso em motores de busca ou aplicações de perguntas e respostas.
- Na tradução automática, o embedding vetorial de uma sentença em um idioma pode ser usado para gerar uma sentença em um idioma diferente com um embedding vetorial semelhante.
- Integrações de sentenças são frequentemente usadas em análise de sentimento. Os classificadores podem ser treinados com exemplos rotulados de cada categoria de sentimento ou utilizando aprendizado supervisionado, classificando novas amostras ao comparar a integração vetorial com a integração aprendida para cada classe. A análise de sentimento também pode ser realizada por meio de zero-shot learning, onde a integração de uma sentença específica é comparada à integração de palavra de uma determinada categorização.

Embeddings de documentos

Embeddings de documentos são usados com frequência para classificar documentos ou páginas da web para indexação em motores de busca ou bancos de dados vetoriais. Modelos típicos para embedding de documentos incluem variantes do BERT, Doc2vec (que é uma expansão do modelo Word2vec) ou outros modelos de embedding de código aberto como o Instructor (link fora de ibm.com).

Outros tipos de embeddings vetoriais

Embora dados de imagem e texto tendam a receber mais atenção, particularmente para casos de uso de IA generativa, uma grande variedade de modalidades de dados pode se beneficiar de embeddings vetoriais.

- **Embeddings de áudio** são usadas para diversas aplicações, desde assistentes de voz a sistemas de recomendação de músicas e sistemas de reconhecimento musical como o Shazam. Elas representam o som por meio das propriedades numéricas dos dados da sua forma de onda. O áudio pode ser incorporado usando [redes neurais recorrentes \(RNNs\)](#), CNNs ou [arquiteturas baseadas em transformadores](#).
- **Integrações de produtos** são frequentemente utilizadas para alimentar sistemas de recomendação em plataformas de e-commerce. Elas geralmente são geradas com algoritmos de [aprendizado não supervisionado](#).
- **Integrações de gráficos** podem ser usadas para modelar e representar estruturas complexas de relacionamento, como redes sociais ou sistemas biológicos. As dimensões de um vetor de integração de gráfico representam como diferentes nós e edges de um sistema estão conectados.

Bancos de dados vetoriais

Bancos de dados tradicionais raramente são otimizados para trabalhar com os dados de alta dimensionalidade comuns em integrações vetoriais. [Bancos de dados de vetores](#), como [IBM®](#)

[watsonx.data](#) são soluções avançadas projetadas para organizar e recuperar objetos de dados em espaços vetoriais de alta dimensionalidade.

Pesquisa vetorial

Um benefício primário de uma solução eficaz de banco de dados vetorial é otimizar a eficiência e precisão das operações de *busca vetorial*: encontrar, classificar e recuperar dados e documentos relevantes por meio da similaridade semântica de seus respectivos embeddings vetoriais com os de seus termos de busca.

Esse tipo de busca por similaridade geralmente é realizado por meio de algoritmos simples de [vizinhos mais próximos](#), que inferem conexões entre pontos de dados com base em sua proximidade no espaço vetorial de alta dimensão.

Busca semântica

A busca semântica usa embeddings vetoriais para alimentar pesquisas que transcendem a correspondência simples de palavras-chave. Por exemplo, retornando resultados para "maçãs" e "laranjas" mesmo que a consulta original tenha sido "fruta".

Geração aumentada de recuperação (RAG)

Este tipo de busca semântica também é usado para possibilitar a [geração aumentada de recuperação \(RAG\)](#), um framework usado para suplementar a base de conhecimento de LLMs sem a necessidade de passar por mais ajuste fino.

No RAG, a busca vetorial é usada para pesquisar fontes de dados externas, ou seja, fontes de dados que não faziam parte dos dados de treinamento de um modelo fundamental e cujas informações, portanto, não poderiam ser refletidas de outra forma na saída do LLM, para *recuperar* informações relevantes e então usar essas informações para *aumentar* as respostas *geradas* pelo LLM.